rqbit 멀티 트래커 지원 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 멀티 트래커 지원 시스템을 상세히 분석합니다. 여러 트래커를 동시에 사용하여 피어 검색의 안정성과 효율성을 높이는 방법을 살펴봅니다.

멀티 트래커 아키텍처

1. 트래커 수집 및 통합

토렌트 파일에서 트래커 추출

```
// Session::add_torrent_internal에서
let mut trackers = torrent
.meta
.iter_announce()
.unique()
.filter_map(|tracker| match std::str::from_utf8(tracker.as_ref()) {
Ok(url) => Some(url.to_owned()),
Err(_) => {
warn!("cannot parse tracker url as utf-8, ignoring");
None
}
```

```
})
.collect::<Vec<_>>();


// 사용자 정의 트래커 추가
if let Some(custom_trackers) = opts.trackers.clone() {
trackers.extend(custom_trackers);
}
```

마그넷 링크에서 트래커 추출

```
trackers: magnet
.trackers
.into_iter()
.filter_map(|t| url::Url::parse(&t).ok())
.collect(),
```

2. TrackerComms 시작 및 관리

멀티 트래커 초기화

```
pub fn start(
info_hash: Id20,
peer_id: Id20,
trackers: HashSet<Url>, // 여러 트래커 URL 집합
stats: Box<dyn TorrentStatsProvider>,
```

```
force_interval: Option<Duration>,

announce_port: u16,

reqwest_client: reqwest::Client,

udp_client: UdpTrackerClient,

) -> Option<BoxStream<'static, SocketAddr>> {

// 지원되는 트래커만 필터링

let trackers = trackers

.into_iter()

.filter_map(|t| match t.scheme() {

"http" | "https" => Some(SupportedTracker::Http(t)),

"udp" => Some(SupportedTracker::Udp(t)),

_ => {

debug!("unsuppoted tracker URL: {}", t);

None

}

})

.collect::<Vec<_>>();

if trackers.is_empty() {

debug!(?info_hash, "trackers list is empty");

return None;

}



tracing::trace!(?trackers);
```

```
// ... 스트림 생성 로직

}
```

3. 동시 트래커 관리

FuturesUnordered를 통한 병렬 처리

```
let s = async_stream::stream! {

use futures::StreamExt;

let comms = Arc::new(Self { /* ... */ });

let mut futures = FuturesUnordered::new();

// 각 트래커에 대해 별도의 Future 생성

for tracker in trackers {

futures.push(comms.add_tracker(tracker, &udp_client))

}

// 모든 트래커를 동시에 모니터링

while !(futures.is_empty()) {

tokio::select! {

// 피어 주소 수신

addr = rx.recv() => {

if let Some(addr) = addr {

yield addr;

}

}
```

```
// 트래커 오류 처리
e = futures.next(), if !futures.is_empty() => {
if let Some(Err(e)) = e {
debug!("error: {e}");
}
}
}
}
};
```

4. 트래커별 독립적 모니터링

HTTP 트래커 모니터링

```
async fn task_single_tracker_monitor_http(&self, tracker_url: Url) -> anyhow::Re
trace!(url=%tracker_url, "starting monitor");
let mut event = Some(TrackerRequestEvent::Started);

loop {
// 지수 백오프를 통한 재시도
let interval = (|| self.tracker_one_request_http(&tracker_url, event))
.retry(
ExponentialBuilder::new()
.without_max_times() // 무제한 재시도
```

```
.with_jitter() // 지터 추가
.with_factor(2.) // 2배씩 증가
.with_min_delay(Duration::from_secs(10)) // 최소 10초
.with_max_delay(Duration::from_secs(600)), // 최대 10분
)
.notify(|err, retry_in| debug!(?retry_in, "error calling tracker: {err:#}"))
.await
.context("this shouldnt fail")?;


event = None;
let interval = self.force_tracker_interval.unwrap_or(interval);
debug!("sleeping for {:?} after calling tracker", interval);
tokio::time::sleep(interval).await;
}
}
```

UDP 트래커 모니터링

```
async fn task_single_tracker_monitor_udp(
&self,
url: Url,
client: UdpTrackerClient,
) -> anyhow::Result<()> {
```

```
let (host, port) = (
url.host().context("missing host")?,
url.port().context("missing port")?,
);


let mut sleep_interval: Option<Duration> = None;
let mut prev_addrs: Option<UdpTrackerResolveResult> = None;
loop {
if let Some(i) = sleep_interval {
trace!(interval=?sleep_interval, "sleeping");
tokio::time::sleep(i).await;
}


// DNS 해석 (이전 결과 캐싱)
let addrs = (async || {
udp_tracker_to_socket_addrs(host.clone(), port)
.instrument(trace_span!("resolve", ?host))
.await
.or_else(|err| prev_addrs.ok_or(err))
})
.retry(
ExponentialBuilder::new()
.without_max_times()
```

```
.with_max_delay(Duration::from_secs(60))
.with_jitter(),
)
.notify(|err, retry| debug!(retry_in=?retry, "error resolving tracker: {err:#}")
.await
.context("this shouldn't happen: failed resolving tracker addrs")?;


prev_addrs = Some(addrs);


// IPv4/IPv6 듀얼스택 지원
match addrs {
UdpTrackerResolveResult::One(addr) => {
match self.tracker_one_request_udp(addr, &client).await {
Ok(sleep) => sleep_interval = Some(sleep),
Err(_) => {
sleep_interval = Some(sleep_interval.unwrap_or(Duration::from_secs(60)))
}
}
}
UdpTrackerResolveResult::Two(v4, v6) => {
// IPv4와 IPv6 동시 요청
let (r4, r6) = tokio::join!(
self.tracker_one_request_udp(v4.into(), &client),
```

```
self.tracker_one_request_udp(v6.into(), &client)
);
sleep_interval = Some(
r4.or(r6)
.ok()
.or(sleep_interval)
.unwrap_or(Duration::from_secs(60)),
)
}
}
}
}
```

5. 트래커 장애 감지 및 자동 전환

개별 트래커 오류 처리

각 트래커는 독립적으로 동작하므로 하나의 트래커가 실패해도 다른 트래커들은 계속 작동합니다:

```
// 트래커별 독립적인 Future
futures.push(comms.add_tracker(tracker, &udp_client))

// 오류 발생 시 해당 트래커만 영향받음
e = futures.next(), if !futures.is_empty() => {
if let Some(Err(e)) = e {
```

```
debug!("error: {e}"); // 로깅만 하고 계속 진행
}
}
```

백오프 전략

```
// HTTP 트래커용 백오프
ExponentialBuilder::new()
.without_max_times() // 무제한 재시도
.with_jitter() // 동시 요청 분산
.with_factor(2.) // 지수적 증가
.with_min_delay(Duration::from_secs(10)) // 최소 지연
.with_max_delay(Duration::from_secs(600)) // 최대 지연

// UDP 트래커용 백오프
ExponentialBuilder::new()
.without_max_times()
.with_max_delay(Duration::from_secs(60))
.with_jitter()
```

6. 트래커 우선순위 관리

암묵적 우선순위

rqbit은 명시적인 트래커 우선순위를 설정하지 않지만, 다음과 같은 암묵적 우선순위가 있습니다:

1. **토렌트 파일의 announce**: 첫 번째 트래커
2. **토렌트 파일의 announce-list**: 추가 트래커들
3. **사용자 정의 트래커**: 설정에서 추가된 트래커들
4. **마그넷 링크 트래커**: 마그넷 링크에 포함된 트래커들

트래커 순서 보존

```
// 토렌트 파일 생성 시 첫 번째 트래커를 announce로 설정
announce: trackers.first().map(|s| s.as_str()).unwrap_or(""),
announce_list: &[trackers],
```

7. 트래커별 통계 수집

세션 레벨 통계

```
pub struct SessionStats {
pub counters: SessionCounters,
pub peers: Arc<AggregatePeerStatsAtomic>,
}

pub struct SessionCounters {
pub blocked_outgoing: AtomicU64,
// ... 기타 카운터들
}
```

트래커 통신 통계

각 트래커는 독립적으로 통계를 수집하지만, 현재 구현에서는 트래커별 세부 통계는 별도로 추적하지 않습니다. 대신 전체적인 피어 발견 통계를 수집합니다.

8. 성능 모니터링 및 최적화

동시 요청 최적화

```
// 각 트래커가 독립적인 태스크로 실행
for tracker in trackers {
futures.push(comms.add_tracker(tracker, &udp_client))
}
```

메모리 효율성

- `Arc<TrackerComms>` 를 통한 공유 상태
- `FuturesUnordered` 를 통한 효율적인 Future 관리
- 채널을 통한 비동기 피어 주소 전달

9. 설정 및 커스터마이징

세션 설정

```
pub struct SessionOptions {
pub force_tracker_interval: Option<Duration>, // 강제 간격 설정
pub disable_trackers: bool, // 트래커 비활성화
pub trackers: HashSet<url::Url>, // 추가 트래커 목록
// ... 기타 설정들
```

```
}
```

토렌트별 설정

```
pub struct AddTorrentOptions {
pub force_tracker_interval: Option<Duration>,
pub trackers: Option<Vec<String>>, // 토렌트별 추가 트래커
// ... 기타 설정들
}
```

10. 트래커 프로토콜 지원

지원되는 프로토콜

```
match t.scheme() {
"http" | "https" => Some(SupportedTracker::Http(t)),
"udp" => Some(SupportedTracker::Udp(t)),
_ => {
debug!("unsuppoted tracker URL: {}", t);
None
}
}
```

프로토콜별 특화 처리

- **HTTP/HTTPS**: `reqwest` 클라이언트 사용, 연결 풀링 지원
- **UDP**: 전용 `UdpTrackerClient` 사용, 연결 ID 캐싱

11. 장애 복구 메커니즘

DNS 해석 실패 처리

```
// 이전 성공한 주소 캐싱
.or_else(|err| prev_addrs.ok_or(err))
```

네트워크 분할 대응

- 각 트래커가 독립적으로 동작
- 하나의 트래커 실패가 전체 시스템에 영향 없음
- 자동 재시도를 통한 일시적 장애 복구

듀얼스택 지원

```
UdpTrackerResolveResult::Two(v4, v6) => {
let (r4, r6) = tokio::join!(
self.tracker_one_request_udp(v4.into(), &client),
self.tracker_one_request_udp(v6.into(), &client)
);
// 둘 중 하나라도 성공하면 사용
sleep_interval = Some(
r4.or(r6)
.ok()
```

```
.or(sleep_interval)
.unwrap_or(Duration::from_secs(60)),
)
}
```

12. 로깅 및 디버깅

구조화된 로깅

```
debug_span!(parent: None, "http_tracker", tracker = %url, info_hash = ?info_hash
debug_span!(parent: None, "udp_tracker", tracker = %url, info_hash = ?info_hash)
```

상세한 오류 로깅

```
.notify(|err, retry_in| debug!(?retry_in, "error calling tracker: {err:#}"))
```

트래커 상태 추적

```
trace!(url=%tracker_url, "starting monitor");
debug!("sleeping for {:?} after calling tracker", interval);
```

결론

rqbit의 멀티 트래커 지원 시스템은 다음과 같은 고급 기능들을 제공합니다:

1. **동시 다중 트래커 지원**: `FuturesUnordered` 를 통한 병렬 처리

2. **프로토콜 다양성**: HTTP/HTTPS와 UDP 트래커 동시 지원

3. **장애 허용성**: 개별 트래커 실패가 전체 시스템에 영향 없음

4. **지능적 백오프**: 지수 백오프와 지터를 통한 효율적인 재시도

5. **듀얼스택 지원**: IPv4와 IPv6 동시 지원

6. **DNS 캐싱**: 이전 성공한 주소 재사용으로 안정성 향상

7. **유연한 설정**: 세션 및 토렌트별 트래커 커스터마이징

8. **포괄적인 로깅**: 구조화된 로깅을 통한 디버깅 지원

이러한 구현을 통해 다양한 네트워크 환경에서도 안정적이고 효율적인 피어 검색을 보장하며, 트래커 장애 시에도 서비스 연속성을 유지합니다.